



BANGLADESH UNIVERSITY OF ENGINEERING & TECHNOLOGY

ANALYSIS REPORT ON OFFLINE-07

COURSE ID: CSE 106

Submitted by :

Raian Rashid Mayel

Level-01, Term-02

Roll: 2405010

Section: A1

Dept: CSE

REPORT ON OFFLINE 07

1. Time Complexity ANALYSIS

The solution uses dynamic programming with a 2D table of size $(m+1) \times (n+1)$, where m and n are the lengths of the input strings. Each cell is computed once.

Segment-Wise Analysis

Segment 1: DP Table Initialization (Base Case)

```
for(int i=0; i<=length1; i++){dp[i][0]=i*deleteCost;} ----- O(m) time complexity
```

```
for(int j=0; j<=length2; j++) {dp[0][j]=j*insertCost;} ----- O(n) time complexity
```

Overall time complexity for this segment: $O(m + n)$

Segment 2: Filling the DP Table (Nested loops)

```
for(int i=1; i<=length1; i++)  
    {for(int j=1; j<=length2; j++){  
        compute dp[i][j] }}
```

Overall time complexity for this segment: $O(m \times n)$

Segment 3: Backtracking

```
while(i>0 || j>0) move based on dp values
```

Overall time complexity for this segment: $O(m + n)$ because at each step, i or j decreases, so total steps are bounded by $m+n$.

Segment 4: Validation Function (For input checking)

```
Check each character to ensure uppercase
```

Overall time complexity for this segment: $O(n)$ because traverses string once.

Final Complexity Summary:

Initialization: $O(m + n)$

DP Computation: $O(m \times n)$

Backtracking: $O(m + n)$

Validation: $O(n)$

Overall time complexity: $O(m \times n)$

2. DP Formulation ANALYSIS

$dp[i][j]$ = minimum cost to convert first i characters of first string to first j characters of second string.

Base Cases

$$dp[i][0] = i \times \text{deleteCost}$$

$$dp[0][j] = j \times \text{insertCost}$$

Transition

If characters match:

$$dp[i][j] = dp[i-1][j-1] \text{ ---match case}$$

else:

$$dp[i][j] = \min(\begin{aligned} & dp[i-1][j] + \text{deleteCost}, \text{---delete case} \\ & dp[i][j-1] + \text{insertCost}, \text{---insert case} \\ & dp[i-1][j-1] + \text{replaceCost} \text{ ----replace case} \end{aligned})$$

Final Recurrence Relation

$$dp[i][j] = \begin{cases} dp[i-1][j-1] & ; \text{if } one[i-1] = two[j-1] \text{ --- match found} \\ \text{minimum of } \begin{cases} dp[i-1][j] + \text{deleteCost} \\ dp[i][j-1] + \text{insertCost} \\ dp[i-1][j-1] + \text{replaceCost} \end{cases} & \text{if } one[i-1] \neq two[j-1] \text{ --- not match} \end{cases}$$

3. Edge Cases ANALYSIS

- Input size is validated to be within 1 to 5000.
- Strings are checked to contain only uppercase letters.

4. Observations regarding sample I/O

Empty string cases are handled via base cases but the assignment states that –

- $1 \leq |S|, |T| \leq 5000$
- Sequences contain uppercase English letters.

The size of both input strings needs to be at least 1 but the sample input 6 and 7 has empty strings as input, that contradicts the question and doesn't follow the constraint.

So, my code doesn't support empty strings as input as given it as constraints. But if the constraint is overthrown, we can use getline methods to take empty strings also.

5. Step-by-Step Example Execution

Example:

First string: ABC

Second string: ADC

Insert cost = 1, Delete cost = 1, Replace cost = 2

Final DP Table:

	" "	A	D	C
" "	0	1	2	3
A	1	0	1	2
B	2	1	2	3
C	3	2	3	2

Minimum Cost = $dp[3][3] = 2$

Operations:

Match A

Replace B with D

Match C
